

Paralelism. Concurență (II)

Conf. Univ. Arthur Molnar

Universitatea Babeș-Bolyai

2025

Privire de ansamblu

- 1 Fire de execuție (continuare)
 - Stările unui fir de execuție
 - Sincronizarea firelor de execuție
 - Mutex
- 2 Pachetul `java.util.concurrent`

Ciclul de viață și stările unui fir de execuție I

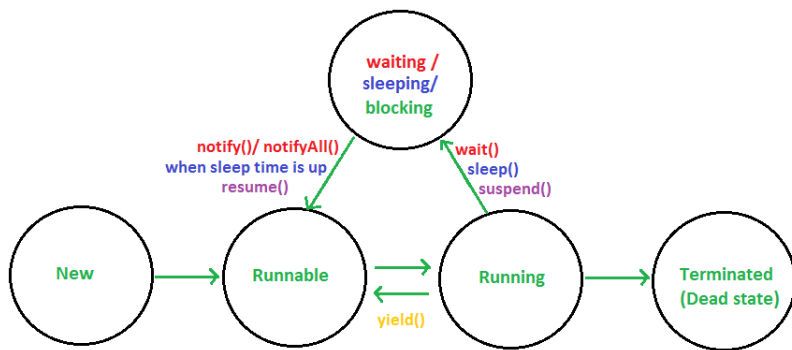


Fig. THREAD STATES

Figure: Sursa figurii: [Stările unui fir de execuție în Java](#)

Ciclul de viață și stările unui fir de execuție II

- **Fir de execuție nou** (eng. *new thread*)
 - obiectul `java.lang.Thread` a fost creat (instanțiat)
 - metoda `start()` nu a fost încă apelată
 - firul de execuție nu a fost lansat în execuție
- **Pregătit de execuție** (eng. *runnable*)
 - Firul de execuție intră în această stare când metoda `start()` este apelată
 - Ce face această metodă:
 - alocă resursele sistem necesare
 - programează executarea codului asociat acestui fir de execuție cu CPU-ul sistemului

Ciclul de viață și stările unui fir de execuție III

- Un fir de execuție poate intra în această stare și după revenirea dintr-o stare de blocat (eng. *blocked*), așteptare (eng. *waiting*) sau o stare de *sleep*.
- Un fir de execuție având starea *pregătit de execuție* este în execuție în mașina virtuală Java, dar e posibil să fie în așteptare pentru eliberarea unor resurse de la sistemul de operare sau procesor.
- **În execuție** (eng. *running*)
 - Firul de execuție a fost activat de componenta *scheduler* a sistemului de operare și se află în execuție
 - Firele de execuție aflate în execuție sunt selectate de sistemul de operare din pool-ul de fire pregătire pentru execuție pe baza priorității fiecăruia.
 - În mod implicit, un fir de execuție își moștenește prioritatea din firul de execuție care l-a creat; prioritatea poate fi însă modificată folosind metoda `setPriority()`
 - Metoda `run()` a obiectului care reprezintă acest fir de execuție este apelată în momentul lansării sale în execuție

Ciclul de viață și stările unui fir de execuție IV

- Cât timp firul este în execuție, metoda `yield()` poate fi utilizată pentru a o trece în starea *pregătit de execuție* (eng. *runnable*)
- **Așteptare/blocat/inactiv** (eng. *Waiting/blocked/sleeping*)
 - Un fir de execuție într-unul din aceste stări este activ dar nu este în execuție, pentru unul din următoarele motive:
 - este *adormit* (eng. *sleeping*) - prin apelarea metodei `sleep()`; firul de execuție va aștepta scurgerea timpului indicat prin parametrul metodei, după care se va întoarce în starea *pregătit de execuție* (eng. *runnable*)
 - este *în așteptare* (eng. *waiting*) - prin apelarea metodei `wait()`; firul de execuție va aștepta ca un alt fir să elibereze obiectul monitor sau lock-ul.
 - este *blocat* (eng. *blocked*) - când încearcă să obțină lock-ul unui obiect, în timp ce acea resursă este deținută de un alt fir de execuție

Ciclul de viață și stările unui fir de execuție V

- **Terminat** (eng. *Dead (terminated)*)

- Un fir de execuție se încheie în mod normal atunci când execuția metodei `run()` se încheie.
- Metoda `stop()` (învechită - eng. *deprecated*, a nu se utiliza) poate încheia execuția unui fir, însă utilizarea sa nu este sigură și astfel nu este recomandată.
- Oprirea corectă a unui fir de execuție se face folosind metoda `interrupt()`
- Un fir își poate termina execuția în cazul apariției unor erori neobișnuite, precum excepții netratate sau erori în gestionarea memoriei (eng. *segmentation fault*)
- Un fir de execuție în această stare nu mai consumă resurse ale CPU-ului

Sincronizarea firelor de execuție I

- Zona de memorie heap e partajată de toate firele de execuție ale programului
- Acest lucru poate cauza probleme în momentul în care același obiect este accesat de mai multe fire de execuție
- Lipsa de sincronizare între operațiile unui program pe mai multe fire de execuție poate duce la coruperea datelor sau alte probleme grave și dificil de reprodus și detectat
- O altă categorie de probleme specifice e reprezentată de ceea ce se numește *deadlock*, atunci când mai multe fire de execuție așteaptă unul după celălalt fără a putea progresa

Sincronizarea firelor de execuție II

- O **secțiune critică** este un bloc de cod ce accesează o resursă partajată între mai multe fire de execuție
- Regula este ca accesul (de regulă în scriere) la resurse partajate să fie limitat, e rând, la câte un singur fir de execuție
- Atunci când un fir de execuție dorește să acceseze o secțiune critică, trebuie să verifice dacă nu cumva un alt fir de execuție îl accesează în mod curent
- Firul de execuție va trebui să aștepte eliberarea resursei critice prin utilizarea unui mecanism de sincronizare

Sincronizarea firelor de execuție III

- Un **monitor** este un mecanism pentru sincronizarea între firele de execuție
- Fiecare obiect în Java este asociat cu un monitor, pe care un fir de execuție îl poate bloca (eng. *lock*) sau debloca (eng. *unlock*)
- La orice moment dat, fiecare monitor poate fi blocat de cel mult un singur fir de execuție
- Alte fire de execuție care încearcă să blocheze monitorul trebuie să aștepte deblocarea acestuia
 - Mecanismul de excluziune mutuală (eng. *mutex*) poate fi utilizat prin **object locks** și permite ca firele de execuție să lucreze independent pe date partajate, fără interferențe nedorite
 - Cooperarea între firele de execuție este posibilă prin folosirea metodelor `wait()` și `notify()`

Sincronizarea firelor de execuție - exemplul producător-consumator (eng. *producer-consumer*) I

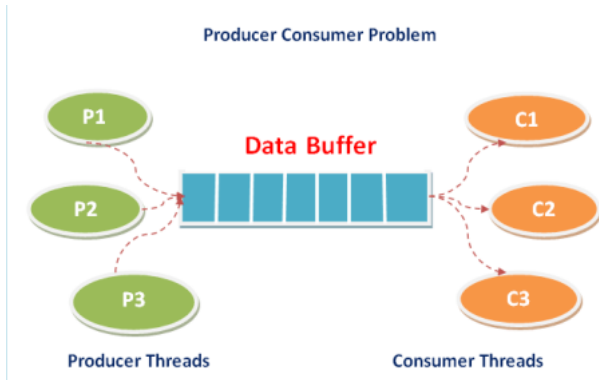


Figure: Sursa figurii: Program producător - consumator

Sincronizarea firelor de execuție - exemplul producător-consumator (eng. *producer-consumer*) II

- Două fire de execuție, un producător și un consumator utilizează aceeași zonă de memorie
- Producătorul adaugă date, iar consumatorul le consumă
- Trebuie să asigurăm că producătorul nu va adăuga date într-o zonă de memorie plină, iar consumatorul nu va citi dintr-o zonă de memorie goală
- O **condiție de cursă** (eng. *race condition*) are loc atunci când mai multe fire de execuție încearcă să acceseze concomitent aceeași resursă

Excluziunea mutuală (eng. *mutex*) I

- Un **mutex** este un mecanism "simplu" de sincronizare care asigură că un singur fir de execuție are acces la o resursă la un moment dat în timp.
- Dacă un fir de execuție apelează o metodă **synchronized** pe un obiect, în primul rând se verifică dacă obiectul este disponibil.
- Dacă **da** $\Rightarrow$ obiectul este marcat ca fiind blocat (eng. *locked*) și firul de execuție începe executarea metodei. La terminarea metodei, obiectul este marcat ca fiind deblocat.
- Dacă **nu** $\Rightarrow$ înseamnă că există un alt fir de execuție care rulează o metodă sincronizată pe acel obiect. În acest caz, firul de execuție așteaptă până obiectul devine din nou disponibil.

Excluziunea mutuală (eng. *mutex*) II

- O metodă de a sincroniza codul rulat în Java este utilizarea cuvântului cheie `synchronized` în semnătura metodei..

```
public synchronized void deposit(double amount)
```

- Când un fir de execuție rulează o metodă sincronizată, nici unui al fir de execuție nu i se permite executarea unei metode sincronizate ce are același obiect ca și "lock".

Comunicarea prin condiții I

- Excluziunea mutuală este necesară pentru rezolvarea problemelor de concurență, însă nu este de regulă suficientă.
- Firele de execuție trebuie să se poată notifica unul pe celălalt în momentul în care au finalizat executarea unei sarcini.
- De exemplu, în cazul în care firul de execuție consumator încearcă să retragă bani dintr-un cont în care nu există fonduri, metoda trebuie să nu facă nimic.
- Ce vrem de fapt în acest exemplu este ca metoda consumator să aștepte momentul în care în cont apar fonduri depuse de un fir de execuție producător.
- Producătorul trebuie să notifice (eng. *notify*) consumatorul (sau consumatorii) de faptul că o sumă de bani a fost depusă.

Comunicarea prin condiții II

- `wait()`:
 - când un fir de execuție apelează această metodă, acel fir de execuție este pus "în așteptare".
 - această metodă blochează firul de execuție până când condiția de continuare este îndeplinită.
 - cât timp firul de execuție este în așteptare, obiectul "lock" pe acest obiect este eliberat, astfel ca obiectul să devină disponibil altor fire de execuție.
- `notifyAll()`:
 - toate celelalte fire de execuție ce sunt blocate în starea de așteptare (folosind metoda `wait()` de mai sus) sunt deblocate și își pot continua execuția.
 - această metodă notifică toate firele de execuție care așteaptă că condiția de continuare a execuției lor a fost îndeplinită.

Interblocaj (eng. *deadlock*)

- Un interblocaj (eng. *deadlock*) poate avea loc în cazul în care un anumit fir de execuție execută o metodă marcată **synchronized** (duce la blocarea obiectului cu pricina), însă executarea nu poate fi finalizată cât timp o anumită condiție nu este îndeplinită.
- Dacă o asemenea condiție este îndeplinită doar când un alt fir de execuție apelează o metodă sincronizată pe același obiect, cele două fire de execuție vor aștepta unul după celălalt fără sfârșit (al doilea fir de execuție nu poate apela cu succes metoda, deoarece obiectul folosit pe post de lacăt este blocat de primul fir de execuție).

java.util.concurrent

- Începând cu versiunea JDK 5.0, mașina virtuală Java a fost îmbunătățită semnificativ pentru programarea concurentă (clasele beneficiază de suportul disponibil în hardware pentru programarea concurentă).
- Pachetul `java.util.concurrent` furnizează metode și structuri de date pentru construirea aplicațiilor rapide și ușor de întreținut, și care utilizează concurența.

Pool-uri de fire de execuție (eng. *Thread pools*) I

- Un mecanism clasic pentru a gestiona un grup mare de taskuri reprezintă combinarea unei cozi de taskuri (eng. *work queue*) cu un pool de fire de execuție.
- Coada de taskuri reprezintă un grup de taskuri ce trebuie procesate.
- Un pool de fire de execuție este o colecție de fire de execuție care scot task-urile din coadă și le execută.
- Un pool de fire de execuție reutilizează fire de execuție create anterior pentru a executa taskurile curente.
- Deoarece firul de execuție deja există în momentul în care preia un task din coada de taskuri, este evitată utilizarea resurselor adiționale pentru crearea unui nou fir de execuție.

Pool-uri de fire de execuție (eng. *Thread pools*) II

- Implementarea corectă a sincronizării firelor de execuție este dificilă în momentul în care mai multe task-uri complexe trebuie să ruleze în paralel, iar rezultatul unora din aceste task-uri depinde de rezultate parțiale obținute pe alte fire de execuție.
- În acest scop, platforma Java oferă interfața `Executor`, care permite crearea de grupuri de fire de execuție, rularea și sincronizarea lor.
- Prin utilizarea mecanismului `Executor` programatorii trebuie să implementeze clasele `Runnable` pe care să le transmită executorului.

```
public interface Executor {  
    void execute (Runnable command);  
}
```

Pool-uri de fire de execuție (eng. *Thread pools*) III

- Un grup de fire de execuție pot fi reprezentate de o instanță a interfeței `java.util.concurrent.ExecutorService`.
- Clasele ce implementează interfața `ExecutorService` furnizează mecanisme pentru pornirea, trimiterea, executarea și încheierea în siguranță a task-urilor.
- Mecanismul executor poate fi de mai multe tipuri:
 - **Executor fix pentru un pool de fire de execuție** (eng. *Fixed thread pool executor*): creează un pool de fire de execuție având un număr fix de fire de execuție care sunt reutilizate.
 - **Executor cu mecanism de cache pentru un pool de fire de execuție** (eng. *Cached thread pool executor*): creează un pool de fire de execuție care va reutiliza firele de execuție în măsura în care acest lucru este posibil, dar poate crea și fire noi de execuție.

Pool-uri de fire de execuție (eng. *Thread pools*) IV

- **Executor cu programare pentru un pool de fire de execuție** (eng. *Scheduled thread pool executor*): creează un pool de fire de execuție care permite programarea executării task-urilor după o anumită durată sau în mod periodic.

Pool-uri de fire de execuție (eng. *Thread pools*) V

- Pentru a opri execuția firelor de execuție rulate prin mecanismul executor se apelează metoda `shutdown()`.
- Aceasta nu va opri imediat executarea taskurilor, dar nu se vor accepta task-uri noi spre executare, iar odată ce toate firele de execuție și-au terminat task-ul curent, `ExecutorService` se va opri.

Pool-uri de fire de execuție (eng. *Thread pools*) VI

- Avantajele utilizării pool-urilor de fire de execuție:
 - Timpul de răspuns se reduce deoarece firele de execuție necesare au fost deja create.
 - Prin înlocuirea implementării de `ExecutorService` utilizată aplicația poate trece de la o aplicație ce folosește un singur fir de execuție la mai multe, sau invers.
 - Stabilitatea sistemului poate fi îmbunătățită prin crearea unui număr de fire de execuție care țin cont în mod dinamic de resursele hardware disponibile.
 - Programatorul nu trebuie să gestioneze în mod manual executarea firelor de execuție.

Clase pentru sincronizare I

• Semafor (eng. semaphore)

- Permite unui anumit număr de fire de execuție accesul la o resursă.
- Utilizează un contor pentru a controla accesul.
- Când un fir de execuție dorește să acceseze resursa protejată, trebuie să obțină permisiunea semaforului.
- Dacă valoarea contorului este > 0 , firul de execuție primește acces, iar valoarea contorului este decrementată.
- Când valoarea contorului este 0, nici un fir de execuție nu primește acces.
- Firele de execuție ce au primit acces trebuie să semnaleze semaforului momentul în care nu mai au nevoie de resursă; în acest moment, semaforul va incrementa valoarea contorului.
- Un mutex este un semafor cu o valoare a contorului egală cu 1.

Clase pentru sincronizare II

- `java.util.concurrent.CyclicBarrier`
 - Permite unui grup de fire de execuție să se aștepte unul pe celălalt pentru a ajunge la un punct comun al execuției, numit și barieră.
 - Un fir de execuție semnalează faptul că a ajuns la barieră prin apelarea metodei `await()`.
 - La apelarea metodei `await()`, execuția firului de execuție este suspendată până când un număr specificat de fire de execuție au apelat aceeași metodă pe barieră.

Clase pentru sincronizare III

- `java.util.concurrent.CountDownLatch`

- Poate fi utilizat pentru a bloca execuția unui fir de execuție până când alte fire de execuție au terminat un anumit task.
- Are un câmp contor care poate fi decrementat folosind metoda `countDown()`.
- Când un obiect de tipul `CountDownLatch` este creat, este specificat și numărul de fire de execuție care vor fi așteptate.
- Fiecare fir de execuție care lucrează va decrementa contorul odată ce și-a terminat taskul.
- În momentul în care contorul atinge valoarea 0, firul de execuție aflat în așteptare poate fi pornit.

Clase pentru sincronizare IV

- `java.util.concurrent.locks.ReentrantLock`
 - Mecanismul de sincronizare prin cuvântul cheie `synchronized` este oarecum rigid:
 - un fir de execuție blochează la intrarea în oricare din metodele marcate ca fiind sincronizate.
 - după eliberarea resursei, oricare fir de execuție o poate accesa - nu există o posibilitate de a aranja firele de execuție într-o coadă de așteptare.
 - asta poate duce la situația în care anumite fire de execuție vor aștepta mult timp pentru a obține accesul la resursă.
 - Mecanismul de `ReentrantLock` este mai flexibil.
 - Permite ca un fir de execuție să obțină resursa protejată de mai multe ori, ceea ce poate preveni interblocajul între firele de execuție.
 - Constructorul clasei acceptă un parametru opțional `fairness` care, dacă este setat `true` prioritizează accesarea resursei de către firul de execuție care a stat în așteptare cea mai mare durată de timp.

Colecții sigure la execuție concurentă (eng. *thread-safe collections*) I

- Codul **sigur la execuție concurentă** este cel care manipulează date în așa fel încât, dacă mai multe fire de execuție lucrează concomitent pe aceleași date, acestea vor fi gestionate în mod corect.
- `Hashtable`, `Vector` - sunt colecții sigure.
- Alte tipuri de colecții sigure pot fi obținute folosind metode ale clasei `Collections`: `Collections.synchronizedMap()`, `Collections.synchronizedList()` și `Collections.synchronizedSet()`.
- Unele colecții mai nou introduse în pachetul `java.util.concurrent` sunt `ConcurrentHashMap`, `CopyOnWriteArrayList` - versiune îmbunătățită a lui `Vector` și `CopyOnWriteArraySet`.

Colecții sigure la execuție concurentă (eng. *thread-safe collections*) II

- Pentru manipularea cozilor au fost introduse interfețele `Queue` și `BlockingQueue`.
- Interfața `Queue` are două implementări:
 - `java.util.concurrent.ConcurrentLinkedQueue<E>` - Acces first-in first-out.
 - `java.util.PriorityQueue<E>`.
- `BlockingQueue` e utilizat pentru a bloca un fir de execuție cât timp anumite operațiuni nu pot fi executate (în cazul unei cozi goale, consumatorii sunt blocați automat până la introducerea unui element de către un producător).
- Această interfață are mai multe implementări: `LinkedBlockingQueue`, `PriorityBlockingQueue`, `ArrayBlockingQueue`, `SynchronousQueue` - acest tip de coadă nu are capacitate, elementele sunt citite de îndată ce au fost adăugate.

javafx.concurrent

- JavaFX furnizează de asemenea un mic cadru de aplicație pentru gestionarea concurenței.
- Furnizează interfața `javafx.concurrent.Worker<V>` care reprezintă un task ce trebuie îndeplinit într-unul sau mai multe fire de execuție.
- Clasa `javafx.concurrent.Task<V>` implementează interfața de mai sus.
- Metoda `call()` a clasei `Task` va executa task-ul (similar cu metoda `run()` a interfeței `Runnable`).